# eBPF-Assisted Machine Learning for Cloud-Native Runtime Security: Telemetry Overhead, In-Kernel Inference, and Containment Trade-offs

Giri Nikhil Vaishnav, Sujit Sai Y, and Shaik Mohammad Asif

***Abstract*—Detectors that learn from eBPF telemetry are widely proposed for container runtime security, but the cost of each pipeline stage and the effect of where the model runs are rarely measured under one protocol. We built a pipeline of cgroup-scoped syscall tracing, ring-buffer transport, count features, a logistic-regression detector and containment through the cgroup v2 freezer. We ran it in five configurations on seven Docker workloads (four benign, three anomalous, one of them held out of training), for 350 randomized runs on one WSL2 host. Telemetry cost 12 to 15 µs per recorded file or process event and about 25 µs per connection event, so throughput loss followed the event rate: nothing measurable at 180 events/s and 54% to 57% at 48,000 events/s. None of 8.87 million events was lost. Running the same model in Q16.16 fixed point inside the eBPF probe added no measurable overhead, agreed with the user-space classifier on every run and raised its alert a median 11.4 µs after the triggering syscall (about 20 µs until user space received it), a time that includes recording that event. The user-space path needed a median 9.1 ms to build features after the run. The detector flagged all 20 trained-anomaly runs with no false positives but missed all 10 runs of the held-out connection burst, because its learned weight on connections was negative. A cgroup freeze took a median 0.044 ms. We also report two silent defects that produced plausible but invalid results in earlier versions, and the tests that now catch them.**

***Index Terms*—eBPF, container security, runtime monitoring, machine learning, cgroup, telemetry overhead, Linux kernel.**

## I. Introduction

Cloud-native applications run as containers that share one host kernel. A compromised container can write and run a payload, read many files or open many connections, and each of these actions reaches the kernel as a system call. Runtime security therefore depends on observing system calls cheaply and acting on them quickly (Jarkas et al., 2025; Minna & Massacci, 2023).

Extended Berkeley Packet Filter (eBPF) has become the standard way to do this. eBPF programs attach to kernel hooks, pass a safety verifier and run without kernel modules (Soldani et al., 2023). Open-source tools such as Falco, Tetragon, Tracee and KubeArmor use eBPF to observe and constrain containers (Her et al., 2025; Syairozi & Arizal, 2025). Rules describe known behaviour, and machine learning (ML) can classify behaviour that is hard to write as a rule. Many recent studies combine eBPF telemetry with ML detectors for containers (R. Kim et al., 2025; Ryu et al., 2026; Shin et al., 2024; Yoon et al., 2026; Zehra et al., 2025).

This combination raises a design question that the 40 primary studies we reviewed do not answer for container system-call telemetry: *where should the model run?* Exporting every event to user space keeps the model flexible but adds transport and scheduling delay. Running the model inside the eBPF program removes that delay, but it limits the model to what the verifier accepts: no floating point, bounded loops and a small stack. In-kernel inference has been studied for network traffic (Hara & Sasabe, 2024; Rohliyanto & Utami, 2026; Taguchi et al., 2026). Hara and Sasabe (2024) compare kernel and user-space placement of the same models for packet classification. We ask the same question for container system-call telemetry, and we add what a security pipeline needs besides a classifier: the cost of the telemetry, the time to a decision and the cost of the response.

We built one pipeline and evaluated it under one protocol. The pipeline records system calls with eBPF tracepoints, transports them through a ring buffer, aggregates them into one-second windows, scores each window with a logistic-regression detector and contains a flagged container by freezing its cgroup. Five configurations differ only in what runs: no instrumentation, telemetry only, user-space floating-point scoring, fixed-point scoring inside the probe, and user-space scoring with containment. Each configuration ran seven workloads (four benign, three anomalous, one of them held out of training) ten times each inside Docker containers, in randomized order, for 350 runs.

We address four research questions. Each corresponds to a gap that our review of the corpus identified (Section III).

- **RQ1 (G1).** How complete is container-scoped eBPF syscall telemetry, and what does it cost the workload?
- **RQ2 (G3).** How do decision latency and overhead change when the same model runs inside the eBPF probe instead of in user space?
- **RQ3 (G2).** How well does a count-based detector separate trained anomalies from benign activity, and does it generalize to an unseen anomaly?
- **RQ4 (G4).** How fast is ML-triggered containment, and does it affect benign workloads?

The paper makes four contributions.

1) **A per-event cost model of syscall telemetry.** Each recorded event added 12 to 15 µs to the workload for file and process events and about 25 µs for connection events. Because throughput loss equals this cost times the event rate, the event rate predicts the overhead across workloads.
2) **A controlled placement comparison.** The same

logistic-regression model, the same telemetry and the same workloads run with scoring in user space and inside the probe. We measure the in-kernel decision latency inside the kernel and compare it with the user-space path.

3) **A measured containment cost and a held-out failure.** Freezing a cgroup took a median of 0.044 ms. The detector missed an unseen attack completely, and the trained weights explain why.
4) **Two measured telemetry defects, the tests that catch them, and open data.** An earlier version of our consumer captured only a fraction of events while its loss counters reported zero loss. We describe the defect, the failure-verified test, and the superseded datasets that we release with the final one.

The paper does not propose a new detector and does not claim production readiness. The workloads are synthetic, the testbed is one host, and classification runs after each workload ends (Section VIII). Section II introduces the mechanisms and the threat model, and Section III reviews related work. Sections IV and V describe the system and the experiments. Sections VI and VII report and discuss the results, and Sections VIII and IX cover threats to validity and conclusions.

## II. Background and Threat Model

### A. eBPF Programs, Maps and the Verifier

An eBPF program is bytecode that the kernel loads, checks and runs when an attached hook fires. Before loading, the in-kernel verifier walks every path through the program. It rejects the program if it could read memory out of bounds, loop without a bound or exceed the limit of one million verified instructions. Each program has a 512-byte stack, and the instruction set has no floating-point operations. Programs keep state in *maps*, which are kernel data structures shared with user space, such as arrays, hash tables and ring buffers.

These rules make eBPF safe enough for production kernels. They also limit what a program can compute. A model that runs inside eBPF needs integer arithmetic, a fixed number of operations and a small, fixed amount of state. Verifier correctness is itself an active research topic (Sun & Su, 2024), and eBPF can be misused as an attack vector (He et al., 2023; Jin et al., 2024; Patel et al., 2026).

### B. Tracepoints and the Ring Buffer

Our probe attaches to syscall tracepoints, which fire when a thread enters a system call, before the kernel executes it. We hook `execve`, `openat`, `connect`, `unlink` and `unlinkat`. A hook on syscall entry sees the call and its caller but not the return value, so it cannot tell whether the call succeeded.

The probe passes events to user space through a BPF ring buffer (Linux 5.8 and later), a single memory region shared by all CPUs. The probe reserves a slot, fills it and submits it. If no slot is free, the reservation fails and the event is lost, and a counter records the failure. In libbpf, `ring_buffer__poll` returns as soon as records are available, not after its timeout. A consumer loop that counts calls instead of measuring elapsed time therefore ends early under load. Section V-G shows that this behaviour broke an earlier version of our pipeline.

### C. Control Groups and the Freezer

Linux control groups (cgroups) organize processes into a hierarchy. Container runtimes such as Docker place each container in its own cgroup. Under cgroup v2, each cgroup is a directory, and its inode number identifies it. An eBPF program reads the identifier of the current task with the helper `bpf_get_current_cgroup_id`, so a probe can keep only the events of one container.

The cgroup v2 freezer stops every process in a cgroup. Writing `1` to the `cgroup.freeze` file asks the kernel to freeze the group, and the file `cgroup.events` reports `frozen 1` once all tasks have stopped. Writing `0` thaws the group. Freezing keeps process state intact, needs no cooperation from the container and can be undone. It is therefore a natural first-stage response for an automated detector.

### D. Fixed-Point Logistic Regression

Logistic regression scores a feature vector $\mathbf{x}$ as $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ and raises an alert when $p \geq 0.5$. Because $\sigma$ is monotonic, this test is equivalent to testing the logit, $\mathbf{w}^\top \mathbf{x} + b \geq 0$. The logit needs only multiplications and additions. In Q16.16 fixed point, each real number $r$ is stored as the integer $\lfloor r \cdot 2^{16} \rceil$. The weights and bias are converted once after training. Feature counts are integers, so the logit is a sum of integer products that the verifier accepts. The only approximation is the rounding of the weights and bias, which we check by comparing fixed-point and floating-point decisions on the same inputs.

### E. Threat Model

We consider an adversary who controls code inside one container on a shared host. The adversary can start processes, read and write files that the container can reach, and open network connections. We model three behaviours that such an adversary commonly shows after gaining a foothold: *drop and execute* (write a payload, run it and delete it), *file sweep* (read many files across system directories, as in reconnaissance) and *connection burst* (open many connections in a short time, as in scanning). Only the connection burst is withheld from training, as a stand-in for an attack the detector has not seen.

We assume that the host kernel and the monitoring components are trusted. The probe and consumer run with root privileges on the host. The adversary cannot load eBPF programs, change the probe or leave the container. We do not consider kernel exploits or container escapes (He et al., 2023; Luo et al., 2025), attackers who imitate benign syscall patterns, or adversarial manipulation of the model.

## III. Related Work

We reviewed 40 primary studies (36 journal articles and 4 peer-reviewed conference papers) and 10 supporting references, published between 2021 and 2026. We extracted each

primary study along five dimensions: the telemetry source, the placement of ML inference, the detection evidence, the enforcement mechanism, and the reporting of overhead and environment. The supporting references cover eBPF safety, cloud-native telemetry and secondary reviews, and we use them for context only. This section follows the five dimensions and ends with the gap that motivates our study.

### A. eBPF Telemetry and Runtime-Security Tooling

Soldani et al. (2023) describe the hook families that eBPF offers on cloud-native platforms: tracepoints, kprobes, traffic control, the eXpress Data Path (XDP) and Linux Security Module (LSM) hooks. Several studies evaluate the open-source tools built on these hooks. Her et al. (2025) compare KubeArmor, Falco, Tetragon and Tracee across Kubernetes security cases. J. Kim and Nam (2024) build a container activity analysis system and compare Tetragon, Falco and Tracee. Syairozi and Arizal (2025) report mean time to detect, detection rate, false-positive rate, CPU and memory for Falco, Tetragon and Tracee on container-escape, denial-of-service (DoS) and cryptomining scenarios. These studies give useful baselines, but they evaluate rule-based tools, not learned detectors.

Other work uses eBPF mainly as a measurement layer. Lim et al. (2021) provide container-granular auditing, and Takagaki and Mizutani (2025) use per-container syscall data to model web-server performance. Among the supporting references, C. Lee et al. (2022) trace microservice packets with eBPF and kernel timestamps, Repetto and Carrega (2022) build a flow monitor for containers, and W. Yang et al. (2024) shorten the service-mesh data path. Together these studies show that eBPF telemetry is practical at container granularity. They rarely quantify how complete the telemetry stream is, that is, whether every relevant kernel event reaches the consumer.

### B. Machine-Learning Detection on eBPF Telemetry

A growing group of studies feeds eBPF telemetry into ML detectors. For containers, Shin et al. (2024) combine syscall monitoring with AdaBoost to detect Dirty COW and Dirty Cred attacks in database containers. R. Kim et al. (2025) detect cryptojacking containers from Tetragon telemetry with several ML models. Ryu et al. (2026) combine host syscall and network-flow telemetry with multiclass ML over ten container scenarios. Zhou et al. (2026) detect abnormal file access with an attention-enhanced autoencoder, and Mitchell et al. (2024) pair an eBPF sensor with an ML ensemble for cloud-native malware. Ajanovic (2026) compare an LSTM autoencoder with traditional ML on Linux host behaviour.

Several detectors target one attack class. Remya et al. (2025) and Zhan et al. (2023) detect application-layer CPU-exhaustion DoS in containers, and Sadiq et al. (2023) detect DoS in Kubernetes from XDP traffic monitoring. Anand et al. (2025) compare tree, forest and support-vector models on DoS traffic captured with eBPF. Outside the container setting, Liu et al. (2026) detect IoT traffic anomalies with an XDP front end and a lightweight deep model, and Peng et al. (2026) detect AIoT gateway anomalies from eBPF timing traces.

The closest systems to an end-to-end pipeline are DeSFAM (Zehra et al., 2025) and the service-mesh system of Yoon et al. (2026). DeSFAM profiles syscalls dynamically, scores risk with a variational autoencoder and an isolation forest, and enforces through BPF-LSM. Yoon et al. (2026) collect syscalls with zero-copy eBPF, apply lightweight online learning per container and isolate compromised workloads. Both report detection and response, but each evaluates one design under its own workloads and metrics. Across this group, detection evidence is mostly within-scenario: training and test data come from the same attack classes. Yoon et al. (2026) show that adaptation to evolving workloads is feasible, but standardized tests under distribution shift remain rare.

### C. Placement of Inference

In the studies above, eBPF serves mainly as the collection layer, and the learned model is a tree ensemble, an autoencoder or a recurrent network. A smaller set places inference or decision logic in the kernel, and these studies concern network traffic. Hara and Sasabe (2024) compare a quantized neural network and a decision tree in kernel space and in user space for packet classification, and expose trade-offs between verifier limits and throughput. Taguchi et al. (2026) run a quantization-aware autoencoder inside eBPF for network intrusion detection. Rohliyanto and Utami (2026) convert an offline random forest into thresholds for XDP pre-filtering on Docker Swarm. S.-Y. Wang and Chang (2022) split an intrusion detection system into in-kernel pattern pre-filtering and user-space rule matching. Packet-processing systems (Miano et al., 2021; You et al., 2022) show that complex processing can run in the kernel with a user-space control plane.

Hara and Sasabe (2024) is the closest to our question, because it compares placements with the models held constant. It addresses packet classification, not container runtime behaviour, and it does not measure the cost of telemetry or of a response. Elsewhere in the corpus, placement is confounded with differences in model, workload and metrics (gap G3 below).

### D. Enforcement and Mitigation

eBPF enforcement is well developed. Gwak et al. (2023) combine Tracee monitoring with BPF-LSM policies for Kubernetes. Cordon (Deng et al., 2025) generates eBPF and LSM policies from deployment files, and BPFGuard (Lu et al., 2025) enforces mandatory access control at syscall, LSM and kernel-function granularity. Optimus (S. Yang et al., 2024), KubeRosy (Her et al., 2024) and Hela (Li et al., 2026) restrict system calls dynamically or across the container lifecycle. BeaCon (Kang et al., 2026) generates container policies through dynamic analysis, and Vaccine (H. Wang et al., 2026) controls processes dynamically to mitigate injection vulnerabilities. At the network layer, Nam et al. (2022) secure inter-container traffic with XDP, Hussain et al. (2025) block IP spoofing in Kubernetes, and Kunerva (S. Lee & Nam, 2023), KubeAegis (B. Kim & Lee, 2024) and ESX (Zhong et al., 2024) derive or unify policies.

Most of these systems enforce policies derived from specifications or rules, not from a learned detector. Where ML drives the response (Yoon et al., 2026; Zehra et al., 2025), the studies report that enforcement works. They report less consistently what it costs: how quickly containment takes effect, whether benign workloads are affected, and whether the action can be reversed.

### *E. eBPF Safety and the Container Threat Model*

Running security logic in the kernel enlarges the trusted computing base. He et al. (2023) show that eBPF itself can be abused for cross-container attacks. Sun and Su (2024) found previously unknown logic bugs in the verifier, BeeBox (Jin et al., 2024) hardens BPF against transient-execution attacks, and KrakenGuard (Patel et al., 2026) proposes fine-grained isolation for eBPF programs. On the threat side, Luo et al. (2025) analyse chained escapes from container to cluster, and Jarkas et al. (2025) survey container exploits and defences. Verifier limits constrain model size, and a kernel-resident model is itself attack surface. Two secondary studies add that reproducibility is weak and benchmarks are missing (Minna & Massacci, 2023), and that standardized evaluation is absent (Kostopoulos et al., 2026).

### *F. Research Gap*

Table I summarizes the studies that combine eBPF telemetry with ML. The components of an eBPF and ML runtime-security pipeline are well studied. Four gaps remain. **G1:** no study measures, under one protocol, how complete the telemetry is, what it costs the workload, how long the decision takes and how long the response takes. **G2:** detection is usually tested on the same attack classes that were used for training. **G3:** kernel and user-space inference are rarely compared with the model, features and workloads held constant, and not for container syscall telemetry. **G4:** the latency, false-positive impact and reversibility of ML-triggered containment are not measured consistently. Our study addresses these gaps with a controlled comparison. It holds the telemetry, features, model and workloads constant and varies only where inference runs and whether a response is taken. It does not propose a new detector.

## IV. System Design

The pipeline has five stages: a kernel probe, a user-space consumer, feature aggregation, a classifier and a staged policy (Fig. 1). All configurations use the same probe, consumer, features and model. They differ only in which stages are active and where the classifier runs.

### *A. Kernel Probe*

The probe is an eBPF program attached to five syscall-entry tracepoints: `execve`, `openat`, `connect`, `unlink` and `unlinkat`. These calls cover process execution, file access, network activity and file deletion, the behaviours in our threat model (Section II-E). A configuration map holds the identifier of the target cgroup, an *anchor* time, the window length and, optionally, the classifier parameters.

On each hit, the probe checks that it is armed, that the calling task belongs to the target cgroup and that the event is not earlier than the anchor time. Events that fail a check are dropped at once, so the probe records only the workload container. For a recorded event, the probe reads the kernel clock, increments a *generated* counter, reserves a 56-byte record in a 4 MiB ring buffer and fills it with the timestamp, a sequence number, the window index, the event type, the process and thread identifiers and the CPU. It then submits the record and increments a *submitted* counter. A failed reservation increments a *reservation-failure* counter instead.

### *B. In-Kernel Classifier*

When inference is enabled, the probe classifies the current window after each recorded event. It keeps five counts for the window in an array map: the numbers of `execve`, `openat`, `connect` and unlink events, and the number of distinct processes. A hash map records the last window in which each process identifier was seen, so that each process is counted once per window. When an event falls into a new window, the counts reset.

The probe then computes the logit $b + \sum_i w_i c_i$ in Q16.16 fixed point, where $c_i$ are the integer counts and $w_i$ and $b$ are the model parameters as Q16.16 integers. If the logit is zero or above, which corresponds to a probability of at least 0.5, and the window has not yet raised an alert, the probe submits a second record of type *alert*. The alert carries the timestamp of the triggering syscall, a second kernel timestamp taken after the decision, and the logit. The difference between the two timestamps is the in-kernel decision latency. Because the probe records the event before it classifies the window, this latency includes the cost of recording the triggering event. The window state is not updated atomically across CPUs, so the counts are exact when the events of a window arrive on one CPU and approximate when several CPUs update them at once.

### *C. User-Space Consumer*

The consumer is a C program built on libbpf. It loads the probe, writes the configuration, attaches the five tracepoints and reports that it is ready. It then waits for a start command from the harness that carries the anchor time, arms the probe and reports that it is armed. From then on, it polls the ring buffer until a wall-clock deadline or a termination signal. For each record, it reads the monotonic clock and writes the record and its receive time as one JSON line. The difference between the receive time and the kernel timestamp is the *transfer latency*. Because `bpf_ktime_get_ns` and `CLOCK_MONOTONIC` use the same clock, it is a direct measurement.

At shutdown, the consumer disarms the probe, drains every record left in the ring buffer, reads the counters and only then detaches the tracepoints. The generated, submitted and consumed counts therefore describe the same events. Section V-G explains why the loop is bounded by elapsed time and not by the number of poll calls.

Table I
PRIMARY STUDIES THAT COMBINE EBPF TELEMETRY WITH MACHINE LEARNING. ENTRIES SUMMARIZE WHAT EACH STUDY REPORTS.

| Study | Role of eBPF | Model | Response and scope |
|---|---|---|---|
| Remya et al. (2025) | Runtime telemetry | Semantic DDoS detector | Detection; semantic DoS in containers |
| Zehra et al. (2025) | Dynamic syscall profiling, BPF-LSM | VAE + isolation forest | BPF-LSM enforcement; containers |
| R. Kim et al. (2025) | Tetragon syscall and flow telemetry | Several ML classifiers | Detection; cryptojacking containers |
| Hara and Sasabe (2024) | XDP packet processing, kernel and user space | Quantized NN, decision tree | Classification; packet IDS |
| Anand et al. (2025) | Kernel packet capture | DT, RF, SVM, TwinSVM | Detection; DoS traffic |
| Taguchi et al. (2026) | In-kernel inference | Quantization-aware autoencoder | Detection; network IDS |
| Shin et al. (2024) | Syscall monitoring | AdaBoost | Detection; container exploits |
| Zhou et al. (2026) | File-access monitoring | Attention autoencoder | Detection; container file access |
| Ryu et al. (2026) | Host syscall and network-flow telemetry | Multiclass ML | Detection; ten container scenarios |
| Rohliyanto and Utami (2026) | XDP pre-filtering with rules from an offline model | Random forest to thresholds | Pre-filtering; Docker Swarm |
| Yoon et al. (2026) | Zero-copy syscall collection | Lightweight online ML | Isolation; service mesh |
| Mitchell et al. (2024) | eBPF sensor | ML ensemble | Detection; cloud-native malware |
| This study | Syscall telemetry; in-kernel scoring (config. D) | Logistic regression, float and Q16.16 | cgroup freeze; containers |

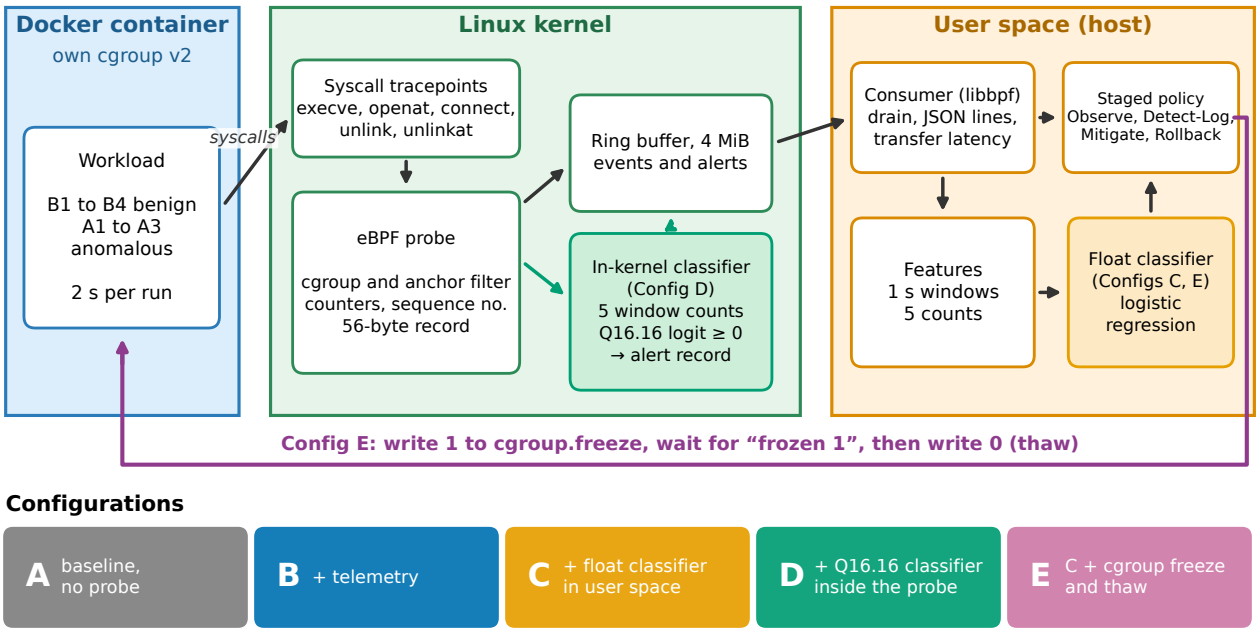



Figure 1. Pipeline and experimental configurations. The probe filters events by cgroup inside the kernel (green). Configuration D also scores each window inside the probe. Configurations C and E score the windows in user space after the run (orange). Configuration E freezes and thaws the container's cgroup (purple).

### D. Features and User-Space Classifier

In user space, the harness groups the events by the window index that the probe assigned and computes the same five counts per window. Windows are one second long and aligned to the anchor time. The harness scores each window with a logistic regression trained on standardized features with the liblinear solver and a threshold of 0.5. After training, we fold the standardization into the model: each effective weight is $w_i/s_i$, and the effective bias is $b - \sum_i w_i \mu_i / s_i$, where $\mu_i$ and $s_i$ are the training mean and standard deviation of feature $i$. Rounding these effective parameters to Q16.16 integers gives the parameters of the in-kernel classifier. The two classifiers therefore apply the same model to the same windows, one in floating point after the run and one in fixed point inside the probe. A run is classified as anomalous if any of its windows is.

### E. Staged Response

The policy has four stages: OBSERVE, DETECT-LOG, CONTROLLED-MITIGATION and ROLLBACK. A score below the threshold keeps the policy in OBSERVE, and a score at or above it moves the policy to DETECT-LOG, which records the alert. Mitigation is disabled by default, and only configuration E enables it. When enabled, the policy moves

to CONTROLLED-MITIGATION and freezes the container's cgroup by writing `1` to `cgroup.freeze`. It then reads `cgroup.events`, with a 0.1 ms sleep between reads, until the kernel reports `frozen 1` or two seconds pass. The policy then enters ROLLBACK at once, writes `0` and waits for `frozen 0`. Immediate rollback lets us measure both transitions in every run and returns the container to its original state.

### F. Configurations

Configuration A is the baseline without instrumentation. B adds the probe and consumer. C adds the user-space floating-point classifier, and D replaces it with the in-kernel fixed-point classifier. E adds the freeze and thaw to C. Fig. 1 summarizes them.

## V. EXPERIMENTAL METHODOLOGY

### A. Testbed

All experiments ran on one host under Windows Subsystem for Linux 2 (WSL2) with Ubuntu 24.04.3 LTS and the Linux kernel 6.6.87.2-microsoft-standard-WSL2 on x86-64. The virtual machine had 20 logical CPUs and about 7.6 GiB of memory. We compiled the probe with Clang 18.1.3 and linked the consumer against libbpf 1.3.0. Docker 29.1.3 ran the workloads, and every workload container used the `python:3.12-slim` image. The harness used Python 3.12.3 and scikit-learn 1.4.1. The probe, consumer and harness ran as root.

### B. Workloads

Table II describes the seven workloads. Each is a function in one Python script that runs for a fixed duration and returns the number of operations it completed. The anomalous workloads implement the three behaviours of the threat model. We designed each of them to differ from the benign workloads in its mix of system calls, not only in file names or arguments, because the probe records only the type of each call (Section V-G).

### C. Run Procedure

Each run follows the same steps. The harness starts a new container without a network interface and with a read-only mount of the project directory, and resolves the container's cgroup path and identifier. For configurations B to E, it starts the consumer with that cgroup, waits until the consumer reports that the probe is attached, sends the start command with the current monotonic time as anchor, and waits until the consumer reports that the probe is armed. Each wait fails the run after five seconds. The harness then runs the workload inside the container with `docker exec` for 2.0 s. The workload measures its own duration and operation count. After the workload ends, the harness reads the container's peak memory from the cgroup, stops the consumer, computes the features and runs the configuration's user-space classifier and policy. In configuration D, the run's decision is the set of alerts that the probe raised during the workload. Configuration E freezes and thaws the still-running container if the run is flagged. Finally, the harness removes the container and writes one JSON record for the run.

### D. Experimental Matrix

The matrix crosses five configurations, seven workloads and ten repetitions, for 350 runs. Repetitions use seeds 0 to 9. The harness shuffled the 350 combinations with a fixed seed (2026), stored the order with the data and ran the combinations in that order, so that slow changes in the host state spread over all configurations.

### E. Calibration

We trained the model once, before the matrix, on separate calibration runs with seed 999. Each workload except A3 ran once for 2 s under the same container and telemetry setup. The resulting 16 windows (10 benign, 6 anomalous) formed the training set. A3 was not run during calibration, so the model never saw a connection burst. We froze the trained model, its fixed-point parameters and its hash before any matrix run. On the training windows, the fixed-point and floating-point logits differed by at most 3.4% and never led to different decisions.

### F. Metrics

- **Throughput overhead.** The change in mean operations per second relative to configuration A for the same workload. Every workload runs for a fixed time, so throughput, not duration, reflects slowdown. B1 performs no operations and is excluded.
- **Per-event cost.** The increase in time per operation in configuration B, divided by the number of recorded events per operation.
- **Telemetry completeness.** The generated, submitted and consumed counts, reservation failures and sequence gaps over all instrumented runs. A gap is a sequence number in the run's range that the consumer never received.
- **Transfer latency.** The per-event time from probe timestamp to consumer receipt, averaged per run.
- **Inference latency.** In configuration C, the time to score a window, measured around the scoring call with Python's nanosecond counter. In configuration D, the time from the kernel timestamp of the triggering syscall to the kernel timestamp taken after the decision, for the first alert of each run, and the time from that decision until the consumer received the alert.
- **Detection.** Precision, recall and F1 on A1 and A2, the false-positive rate on each benign workload, and recall on the held-out A3.
- **Containment.** In configuration E, the time from the freeze request until the kernel reports the cgroup as frozen, the corresponding thaw time, and whether the kernel confirmed each transition.

### G. Telemetry Validation

Two defects in earlier versions of this pipeline would have invalidated its results without any visible error. We describe them because both are easy to repeat.

Table II
WORKLOADS. EACH RUNS FOR 2 S INSIDE A FRESH CONTAINER. THE LAST COLUMN DEFINES THE OPERATION THAT IS COUNTED FOR THROUGHPUT.

| ID | Class | Behaviour | Operation |
|---|---|---|---|
| B1 | Benign | Idle: sleeps for the whole duration | None |
| B2 | Benign | File processing: writes and reads back a 768-byte file from a set of 32 names in a temporary directory, deletes every fourth file | One write, read and optional delete |
| B3 | Benign | Process churn: starts the `true` program and waits for it, repeatedly | One process |
| B4 | Benign | Local client and server: a request and a reply over loopback TCP with a 10 ms pause | One round trip |
| A1 | Anomalous | Drop and execute: writes a shell script, marks it executable, runs it and deletes it | One payload run |
| A2 | Anomalous | File sweep: reads the first 1,024 bytes of every file under `/etc` and the Python library tree, read only | One file opened |
| A3 | Anomalous (held out) | Connection burst: opens loopback TCP connections to a local sink with no pause | One connection |

The first was in the consumer. Its main loop called `ring_buffer__poll` with a timeout a fixed number of times, intending to run for a set duration. Under load, the call returns as soon as records are available, so the loop finished in about 0.1 s. The consumer then stopped while the workload was still running. Its counters still showed zero reservation failures and zero sequence gaps, because every event that the probe did see was delivered. Only the event totals revealed the problem. A test that runs 100 executions of `/bin/true` inside a test cgroup captured between 11 and 33 `execve` events with the old loop and 101 with the corrected loop, which is bounded by elapsed time. We confirmed that the test fails when the old loop is restored and passes with the fix. Three further integration tests run against the live kernel: one checks the cgroup filter, one checks that the in-kernel classifier alerts exactly when its logit reaches zero, and one checks that the consumer stays alive under event load.

The second defect was in the workloads. In an earlier version, the anomalous file workload called the same function as the benign file workload with a different seed, which changed only file names, and the anomalous execution workload started `true` exactly as the benign process-churn workload did. The probe records only syscall types, so each pair was indistinguishable to the classifier, yet the training data labelled the pairs differently. The detection scores of that version reflected the labels and not the behaviour. We replaced both anomalous workloads with the designs in Table II. The datasets from before each correction are released with the final one (Data Availability).

## VI. RESULTS

All results come from the 350 run records of the final dataset. We report means with sample standard deviations over the ten repetitions of each cell, unless stated otherwise.

### A. Telemetry Completeness (RQ1)

Across the 280 instrumented runs (configurations B to E), the probe recorded 8,870,971 events from the workload containers. All of them were submitted to the ring buffer and all were consumed. There were no reservation failures and no sequence gaps. The probe filters by cgroup in the kernel, so every recorded event belongs to a workload. Before the matrix, the capture test counted 101 `execve` events for 100 executions of `/bin/true`, which meets its pass criterion of at least 100. At the event rates of our workloads, up to about 48,000 events per second for A2, the pipeline delivered every event that the probe recorded.

### B. Throughput Overhead and Per-Event Cost (RQ1)

Table III and Fig. 2 show the throughput of each workload. The overhead differs by more than two orders of magnitude across workloads. B4, which makes about 180 monitored calls per second, showed no measurable loss. The file workloads lost the most: B2 lost 43% to 47% and A2 lost 54% to 57%. The process and connection workloads lost between 9% and 27%.

During the workload, configurations B, C and E run identical code, and D differs only in that the probe also evaluates the classifier. The spread within each row of Table III is at most 6.3 percentage points (A3), and D is not consistently slower than B. In-kernel inference therefore added no overhead that we could distinguish from run-to-run variation.

The overhead tracks the event rate, not the type of workload. In configuration B, the throughput loss equals the per-event cost multiplied by the recorded event rate, so each workload lies on a line through the origin whose slope is its per-event cost (Fig. 3). For the four file and process workloads, this cost lies between 11.8 and 14.9 µs per event (A2: 11.8, A1: 13.0, B3: 14.4, B2: 14.9). The connection burst A3 costs 24.9 µs per event. The estimate for B4 is not meaningful, because its overhead is within noise.

Memory overhead was small. The consumer's peak resident memory was 10.3 MB in every instrumented configuration. The peak memory of the workload container was 18.8 MB in configuration A and between 18.2 and 18.6 MB in configurations B to E.

### C. Decision Latency (RQ2)

Fig. 4 summarizes the latency of each stage. In configuration D, the probe decided a median of 11.4 µs after the kernel timestamp of the triggering syscall (95th percentile 16.8 µs, maximum 19.1 µs), measured inside the probe for the first alert

Table III
THROUGHPUT IN OPERATIONS PER SECOND (MEAN ± STANDARD DEVIATION OVER 10 RUNS) AND CHANGE RELATIVE TO CONFIGURATION A. EVENT RATE IS THE MEAN NUMBER OF RECORDED EVENTS PER SECOND IN CONFIGURATION B. PER-EVENT COST IS THE INCREASE IN TIME PER OPERATION IN B DIVIDED BY THE EVENTS PER OPERATION. B1 PERFORMS NO OPERATIONS AND IS OMITTED.

| Workload | Events/s | Cost (µs/event) | A (ops/s) | B | C | D | E |
|---|---|---|---|---|---|---|---|
| B2 file processing | 28,885 | 14.9 | 22,508 ± 1,304 | −43.2% | −45.1% | −46.6% | −46.2% |
| B3 process churn | 11,783 | 14.4 | 2,010 ± 122 | −16.9% | −12.2% | −15.2% | −15.7% |
| B4 client-server | 180 | n/a | 88 ± 1 | +0.0% | +0.6% | +1.1% | −0.6% |
| A1 drop and execute | 8,980 | 13.0 | 1,677 ± 147 | −11.7% | −8.9% | −14.7% | −9.1% |
| A2 file sweep | 48,114 | 11.8 | 96,615 ± 8,938 | −56.8% | −55.1% | −53.9% | −53.8% |
| A3 connection burst | 10,806 | 24.9 | 14,659 ± 1,314 | −26.9% | −21.8% | −20.6% | −23.0% |

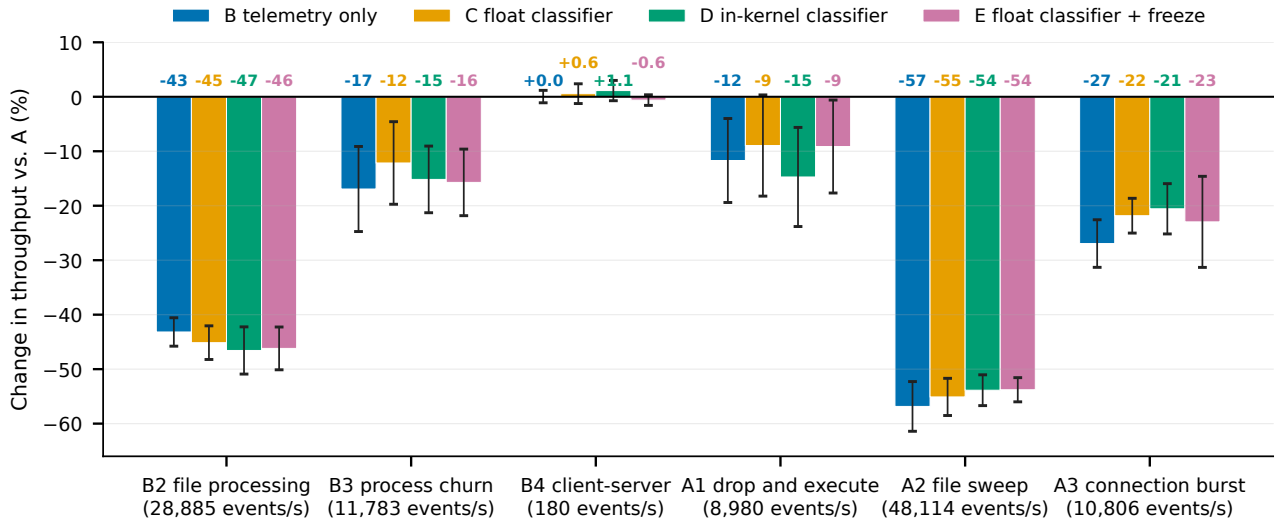



Figure 2. Change in throughput relative to configuration A (mean of 10 runs). Whiskers show one standard deviation of the run values, relative to the mean of A. The numbers above the axis give the mean change. The recorded event rate of each workload (configuration B) is shown below its name.

of each of the 20 flagged runs. This time includes recording the triggering event. The alert then reached the consumer a median of 8.7 µs later. A decision made in the kernel was therefore available in user space about 20 µs after the syscall that caused it.

In configuration C, the decision depends on a whole window. Each event reached the consumer with a median transfer latency of 21.3 µs (95th percentile 106.7 µs, maximum 137.2 µs). After the run, building the windows from the events took a median of 9.1 ms (95th percentile 42.6 ms, maximum 49.6 ms), and scoring a window took a median of 26.3 µs in Python (95th percentile 95.1 µs, maximum 191.5 µs).

The in-kernel and user-space classifiers reached the same decision on every run (Table IV), although the fixed-point weights are coarse: the weight of the `openat` count rounds to the integer 3 in Q16.16 units.

### D. Detection (RQ3)

Table IV shows how many runs each configuration flagged. All three classifier configurations flagged all 20 runs of the trained anomalies A1 and A2 and none of the 40 benign runs. On the trained workloads, precision, recall and F1 are therefore 1.0, and the false-positive rate is 0 for every benign workload. The in-kernel classifier raised no alert on any benign run. The held-out connection burst A3 was never flagged, so its recall is 0.

Table IV
RUNS FLAGGED AS ANOMALOUS, OUT OF 10 PER CELL. A3 WAS HELD OUT OF TRAINING.

| Config | B1 | B2 | B3 | B4 | A1 | A2 | A3 |
|---|---|---|---|---|---|---|---|
| C | 0 | 0 | 0 | 0 | 10 | 10 | 0 |
| D | 0 | 0 | 0 | 0 | 10 | 10 | 0 |
| E | 0 | 0 | 0 | 0 | 10 | 10 | 0 |

In configuration C, the highest score of any benign run was 0.420, and the lowest score of any trained anomaly was 0.585, against a threshold of 0.5. Every A3 run scored below 0.001 (Fig. 5a). The trained weights show why (Fig. 5b).

### E. Containment (RQ4)

Configuration E triggered containment in all 20 runs that it flagged (A1 and A2) and in no benign run. The kernel confirmed the freeze, and the thaw succeeded, in every one of these runs. The latencies below are those of the last freeze and thaw in each run. The median time from the freeze request until the kernel reported the cgroup as frozen was 0.044 ms (95th percentile 0.365 ms, maximum 0.401 ms). The median thaw time was 0.041 ms (95th percentile 0.061 ms, maximum

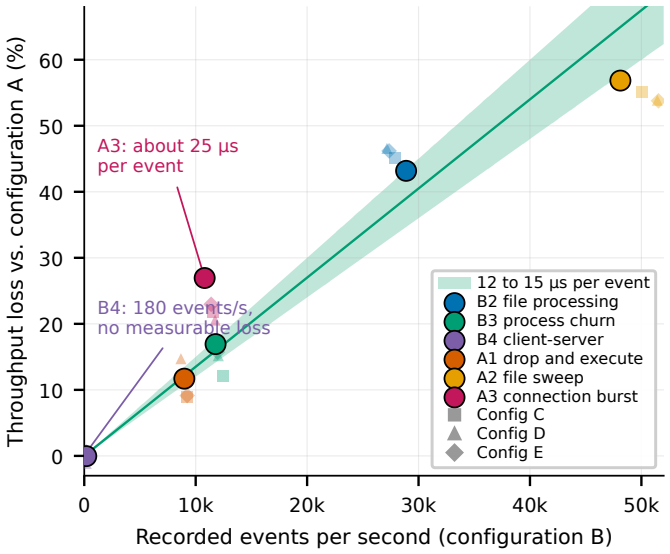



Figure 3. Throughput loss against recorded event rate. Large markers show configuration B, small markers show C, D and E. The band is the loss predicted by a fixed cost of 12 to 15 µs per event. A3 lies above the band at about 25 µs per event.

0.066 ms). Counting from the end of the workload, the median time to a confirmed freeze, which adds feature aggregation and scoring, was 24.4 ms, and the freeze itself was a median 0.5% of it (Fig. 6). These freezes happened after the workload had finished, when the only process left in the container was its idle entry process.

## VII. Discussion

### A. Telemetry Cost Is a Per-Event Cost

The main cost of the pipeline is not the model. It is recording each event. On our testbed, every recorded file or process event added 12 to 15 µs to the workload, and every connection event about 25 µs. The overhead of a workload is roughly its event rate multiplied by this cost. A service that makes a few hundred monitored calls per second, like B4, pays nothing measurable. A workload that opens tens of thousands of files per second, like A2, loses more than half of its throughput.

The in-kernel timestamps suggest where this time goes. For the events that raised an alert, the probe's own execution, from its first clock read to the clock read after the decision, took a median of 11.4 µs. That is the same magnitude as the per-event overhead, which indicates that most of the cost is spent inside the probe and not in the consumer. Configurations B and D show the same overhead, so the cost does not come from the classifier. The remaining candidates are the syscall-tracepoint path, the ring-buffer reservation and submission, and the virtualization layer of WSL2. We have not separated them, and the cost may be lower on bare metal. Whatever its source, a design that records every event scales poorly with the event rate, and reducing the number of recorded events by filtering or counting earlier attacks the dominant cost directly.

### B. What In-Kernel Inference Changes

Moving the classifier into the probe had three effects. First, it added no overhead that we could distinguish from run-to-run variation. The probe already paid for recording each event, and a five-term integer sum and a hash-map update are small in comparison. Second, it changed no decision. The fixed-point classifier agreed with the floating-point classifier on every run, even though some of its weights round to very small integers. Third, it changed when a decision becomes available. The probe evaluates the classifier after every event and alerts at the first event that pushes the logit across the threshold, about 20 µs before user space learns of it. A user-space classifier that scores complete windows must wait at least until the window closes, one second in our design, and then spend milliseconds building features.

The latency advantage therefore does not come from faster arithmetic. It comes from incremental evaluation inside the event path, which lets the decision happen mid-window. This advantage only matters if the response is equally fast. In our enforcement configuration, the freeze is still requested from user space after the run. A design that acts on the in-kernel alert at once, either from the consumer or from the kernel itself, would turn the 20 µs decision into a response within tens of microseconds. Our measurements of each stage bound what such a design can achieve.

The coarse fixed-point weights are a warning for larger models. With integer counts in the thousands, small effective weights round to a few units in Q16.16, and the logit carries an error of a few percent. Our decisions were unaffected because the margins between classes were wide. Models with finer decision boundaries would need scaled features or more fractional bits.

### C. Detection on Synthetic Workloads

The detector separated the two trained anomalies from all four benign workloads without error. This shows that the pipeline works end to end. It does not show that a count-based logistic regression detects real attacks. The workloads are synthetic, and each was designed to have a distinct mix of system calls, so their per-window counts differ by orders of magnitude and a linear model separates them easily.

The held-out workload is the more informative result. A3 opens 11,000 to 15,000 connections per second, depending on the configuration, an obvious anomaly to a human analyst, yet every A3 run scored below 0.001. The trained weights explain why. The only training workload that made connections was the benign client and server B4, so the model learned a negative weight for the `connect` count (standardized coefficient $-0.63$). More connections pushed the score down, and the connection burst looked more benign, not less. The model learned which counts distinguished its training classes, not what makes behaviour anomalous. This is the distribution-shift gap (G2) in its simplest form: a supervised detector trained on a few attack types should be expected to miss new ones, and evaluations that test only on trained attack types overstate detection.

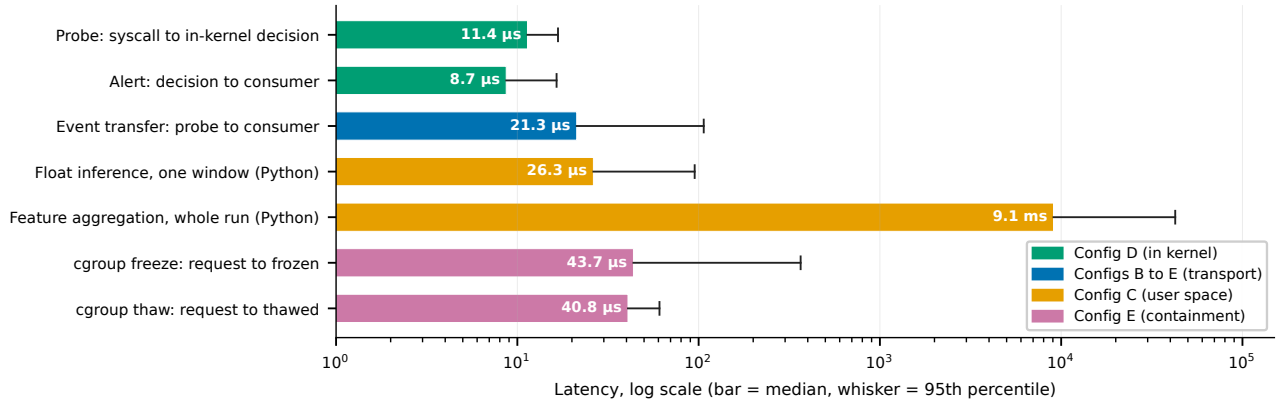



Figure 4. Latency of each stage of the decision path on a logarithmic scale. Bars show the median and whiskers the 95th percentile. The in-kernel decision time includes recording the triggering event. Feature aggregation covers the whole run.

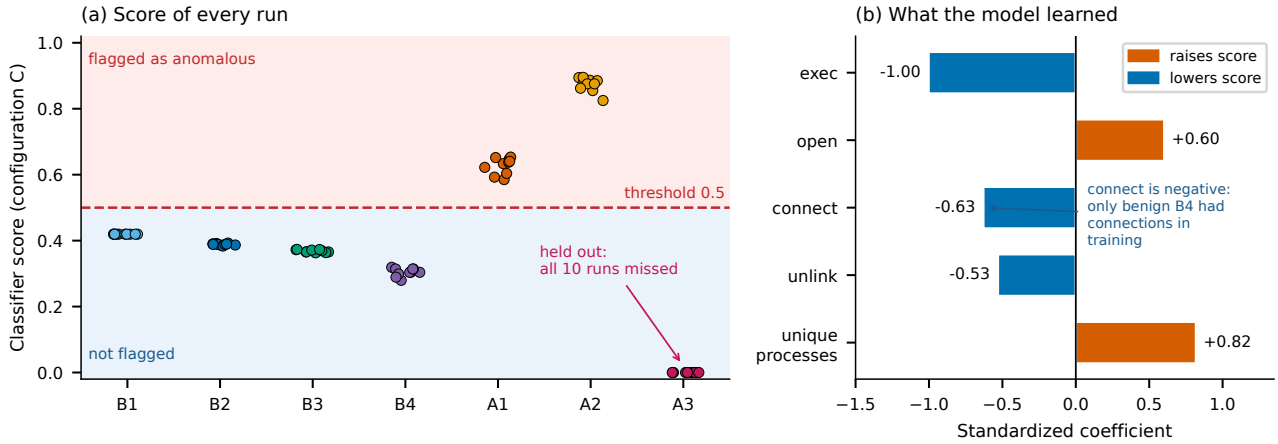



Figure 5. Detection. (a) The score of every run in configuration C against the threshold of 0.5. Blue, green and purple points are benign workloads, orange and yellow points are trained anomalies, and magenta points are the held-out connection burst. (b) The standardized coefficients of the frozen model.

### *D. Containment Through the cgroup Freezer*

Freezing a container through cgroup v2 was fast, reliable and reversible in our runs. Every freeze took effect in well under a millisecond, the kernel confirmed every one, and every container thawed cleanly. The freeze itself was a median 0.5% of the time to a confirmed freeze, so the delay before containment lies in feature processing, not in the freezer. The detector raised no false positives, so no benign container was frozen, and we could not measure the cost of a wrong freeze.

Two properties make the freezer a good first-stage response for an ML detector. It needs no knowledge of the container's internals, and it can be undone if the alert proves false, which limits the damage of false positives compared with killing the container. Its speed in our runs is, however, a lower bound. Each freeze happened after the workload had ended, when only an idle process remained. Freezing a container with many active threads, some of them inside system calls, is likely to take longer.

### *E. Lessons for Evaluating eBPF Pipelines*

The two defects in Section V-G share a pattern. Each produced plausible numbers and passed the checks in place at the time. The consumer reported zero loss while capturing only a fraction of events, and the classifier reported meaningful scores while separating workloads that were identical to the probe. Loss counters measure the transport, not the coverage, and labels measure intent, not behaviour. We recommend that evaluations of eBPF security pipelines include a capture test with a known number of events, confirm that the test fails on a known-bad consumer, and check that labelled workload classes differ in the features the probe can observe.

## VIII. Threats to Validity

**Internal validity.** We randomized the run order, but differences of a few percentage points between configurations remain within run-to-run variation, which reached 6.3 points for A3. Overhead differences smaller than this should not be interpreted. Configurations C and E classified and responded after each workload ended, so their decision and freeze times do not include the time an online detector would need. The model was trained on 16 windows from one calibration run per workload. The in-kernel window state is not updated atomically, so concurrent events on several CPUs can make the in-kernel counts differ slightly from the user-space counts. The in-kernel decision latency rests on 20 values, one per flagged run.

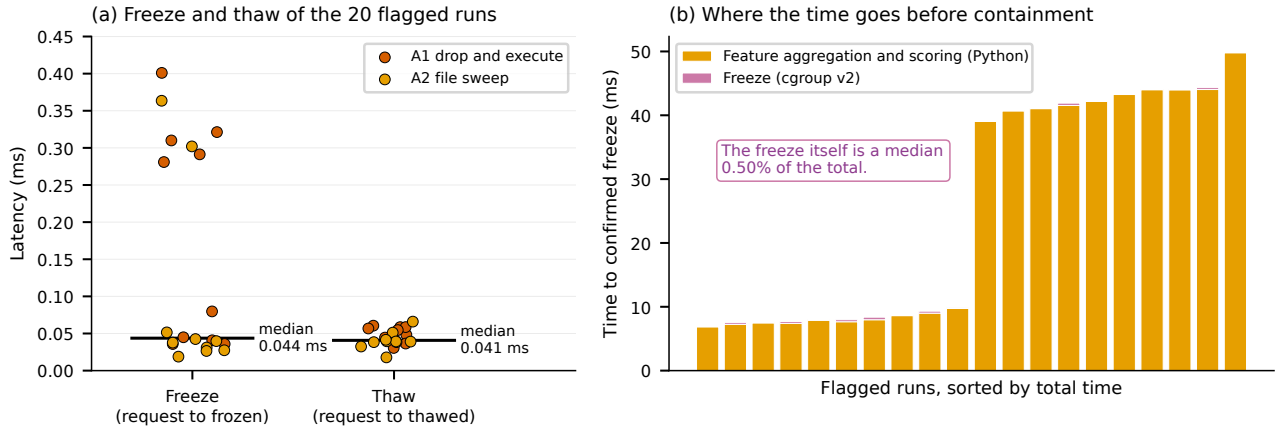



Figure 6. Containment in configuration E. (a) Freeze and thaw latency of the 20 flagged runs, with the median as a black line. (b) The time to a confirmed freeze for each flagged run, split into feature aggregation and scoring (orange) and the freeze itself (purple).

**Construct validity.** Throughput is measured in operations per workload, and an operation means different things in different workloads, so overheads are comparable within a workload but not across workloads. The transfer latency includes the time a record waits for the consumer, not only the time to cross the kernel boundary. The in-kernel decision latency includes the cost of recording the triggering event and of reserving the alert record. The user-space inference latency was measured in Python. The freeze latency was measured on containers whose workload had finished, and each freeze was timed with a polling loop that reads `cgroup.events` with a 0.1 ms sleep, so latencies near that interval carry corresponding resolution limits.

**External validity.** All runs used one host, one kernel version and the WSL2 virtualization layer, and the per-event cost may differ on bare metal or other hypervisors. The workloads are synthetic and short, run one at a time, and use Docker without an orchestrator. The probe hooks five system calls on entry. It does not see `execveat`, `open`, renames or the results of the calls it hooks, so it would miss behaviour that uses only those paths. The detector is a single linear model trained on two attack types, and we make no claim about real attacks, adaptive attackers or other model families. We did not compare against existing tools such as Falco or Tetragon.

**Conclusion validity.** Each cell has ten runs. We report means, standard deviations and ranges and avoid claims that depend on small differences. We did not apply significance tests, because the main effects, such as overheads above 40% and the complete miss of A3, are far larger than the observed run-to-run variation, and the absence of a measurable cost for in-kernel inference is a statement about differences that are indistinguishable from that variation.

## IX. Conclusion

We evaluated an eBPF and machine-learning runtime-security pipeline for containers under one measurement protocol, holding telemetry, features, model and workloads constant across five configurations and 350 runs in randomized order. Five findings stand out. First, after we corrected a consumer defect, the telemetry was complete: 8.87 million events with no loss. Second, its cost was a per-event cost of 12 to 25 µs on our testbed, so overhead ranged from nothing measurable to 57%, depending on how many monitored system calls a workload made. Third, running the same model inside the probe in Q16.16 arithmetic added no measurable overhead, changed no decision and made the decision available in user space about 20 µs after the triggering syscall, instead of after a full window. Fourth, a detector that was perfect on its trained anomalies missed every run of an unseen connection burst, because it had learned that connections indicate benign activity. Fifth, containment through the cgroup v2 freezer took effect in well under a millisecond and was always reversible.

These results suggest that, for eBPF-based ML, the value of kernel placement lies in incremental, per-event decisions inside the event path, not in faster arithmetic. Future work should act on in-kernel alerts directly so that containment reaches running workloads within microseconds, enlarge the training set, add realistic benign workloads and real attack tools, compare against existing runtime-security tools, and repeat the study on bare-metal hosts and under Kubernetes.

## Data Availability

The code, raw run records, processed tables and figures are available at https://github.com/SujitSaiY2007/Computer-Networks-Case-Study-. The results in this paper come from commit `d0870ab`. The run records are in `experiments/raw`, and the run order is in `experiments/manifests/run_order.json`. The datasets produced before each correction are kept, with their calibration data, in `experiments/raw_v1_early_exit_loader`, `experiments/raw_v2_same_code_anomalies` and `experiments/raw_v3_python_configD`, so that the effect of each change can be inspected.

## References

Ajanovic, M. (2026). Behavioral anomaly detection in Linux systems using eBPF and LSTM neural networks: A comparative study with traditional machine learning. *International Journal of Computer Applications*, *187*(99), 1–6. https://doi.org/10.5120/ijca502db8d4e336

Anand, N., Saifulla, M. A., Aakula, P. K., Ponnuru, R. B., & Reddy, C. R. P. (2025). Enhancing intrusion detection against denial of service and distributed denial of service attacks: Leveraging extended Berkeley packet filter and machine learning algorithms. *IET Communications*. https://doi.org/10.1049/cmu2.12879

Deng, Q., Xu, Z., Zhou, Q., & Zhang, Y. (2025). Cordon: Enhancing security through kernel-level control in containerized computing environments. *Computers & Security*, *151*, 104644. https://doi.org/10.1016/j.cose.2025.104644

Gwak, S., Doan, T.-P., & Jung, S. (2023). Container instrumentation and enforcement system for runtime security of Kubernetes platform with eBPF. *Intelligent Automation & Soft Computing*. https://doi.org/10.32604/iasc.2023.039565

Hara, T., & Sasabe, M. (2024). Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree. *Computer Networks*, *243*, 110188. https://doi.org/10.1016/j.comnet.2024.110188

He, Y., Guo, R., Xing, Y., Che, X., Sun, K., Liu, Z., Xu, K., & Li, Q. (2023). Cross container attacks: The bewildered eBPF on clouds. *Proceedings of the 32nd USENIX Security Symposium*.

Her, J., Jo, C. H., Park, T., & Lee, S. (2024). KubeRosy: A dynamic system call filtering framework for containers. *IEEE Access*. https://doi.org/10.1109/ACCESS.2024.3486772

Her, J., Kim, J., Kim, J., & Lee, S. (2025). An in-depth analysis of eBPF-based system security tools in cloud-native environments. *IEEE Access*, *13*. https://doi.org/10.1109/ACCESS.2025.3605432

Hussain, A., Aziz, A., Syed, H. J., & Raza, S. (2025). Preventing IP spoofing in Kubernetes using eBPF. *Computers, Materials & Continua*. https://doi.org/10.32604/cmc.2025.062628

Jarkas, O., Ko, R., Dong, N., & Mahmud, M. R. (2025). A container security survey: Exploits, attacks, and defenses. *ACM Computing Surveys*, *57*(7), 170. https://doi.org/10.1145/3715001

Jin, D., Gaidis, A. J., & Kemerlis, V. P. (2024). BeeBox: Hardening BPF against transient execution attacks. *Proceedings of the 33rd USENIX Security Symposium*.

Kang, H., Marin, E., You, M., Perino, D., Shin, S., & Kim, J. (2026). BeaCon: Automatic container policy generation using environment-aware dynamic analysis. *Computers & Security*, 104789. https://doi.org/10.1016/j.cose.2025.104789

Kim, B., & Lee, S. (2024). KubeAegis: A unified security policy management framework for containerized environments. *IEEE Access*. https://doi.org/10.1109/ACCESS.2024.3487990

Kim, J., & Nam, J. (2024). eBPF-based container activity analysis system. *Transactions of the Korea Information Processing Society*, *13*(9), 404–412. https://doi.org/10.3745/TKIPS.2024.13.9.404

Kim, R., Ryu, J., Kim, S., Lee, S., & Kim, S. (2025). Detecting cryptojacking containers using eBPF-based security runtime and machine learning. *Electronics*, *14*(6), 1208. https://doi.org/10.3390/electronics14061208

Kostopoulos, S., Tsakonas, P., & Markakis, E. K. (2026). eBPF-based cybersecurity mechanisms: A systematic literature review. *International Journal of Information Security*, *25*, 150. https://doi.org/10.1007/s10207-026-01326-x

Lee, C., Yoshitani, R., & Hirotsu, T. (2022). Enhancing packet tracing of microservices in container overlay networks using eBPF. *Proceedings of ACM AINTEC 2022*. https://doi.org/10.1145/3570748.3570756

Lee, S., & Nam, J. (2023). Kunerva: Automated network policy discovery framework for containers. *IEEE Access*. https://doi.org/10.1109/ACCESS.2023.3310281

Li, S., Zhou, J., Li, X., Meng, W., Gong, B., & Hu, J. (2026). Hela: A system call restriction framework for protecting the entire containers lifecycle. *IEEE Transactions on Cloud Computing*. https://doi.org/10.1109/TCC.2026.3677796

Lim, S. Y., Stelea, B., Han, X., & Pasquier, T. (2021). Secure namespaced kernel audit for containers. *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. https://doi.org/10.1145/3472883.3486976

Liu, S., Law, K. L. E., Zhang, Z., & Lam, C. T. (2026). Real-time eBPF/XDP-based deep learning framework for anomaly traffic detection in Internet of Things networks. *IEEE Internet Computing*. https://doi.org/10.1109/MIC.2026.3692726

Lu, H., Du, X., Hu, D., Su, S., & Tian, Z. (2025). BPFGuard: Multi-granularity container runtime mandatory access control. *IEEE Transactions on Cloud Computing*. https://doi.org/10.1109/TCC.2025.3551838

Luo, J.-N., Zou, W.-H., & Huang, S.-W. (2025). From container to cluster: Chained escape attacks in Kubernetes and orchestration platforms. *IEEE Access*. https://doi.org/10.1109/ACCESS.2025.3627543

Miano, S., Doriguzzi-Corin, R., Risso, F., Siracusa, D., & Sommese, R. (2021). A framework for eBPF-based network functions in an era of microservices. *IEEE Transactions on Network and Service Management*. https://doi.org/10.1109/TNSM.2021.3055676

Minna, F., & Massacci, F. (2023). SoK: Run-time security for cloud microservices. are we there yet? *Computers & Security*, *127*, 103119. https://doi.org/10.1016/j.cose.2023.103119

Mitchell, B. S., Chandnani, A., Carter, J., Roumelioti, D., & Mancoridis, S. (2024). Malware detection in cloud native environments. *Proceedings of the 2024 Artificial Intelligence and Cloud Computing Conference (AICCC 2024)*. https://doi.org/10.1145/3719384.3719465

Nam, J., Lee, S., Porras, P., Yegneswaran, V., & Shin, S. (2022). Secure inter-container communications using XDP/eBPF. *IEEE/ACM Transactions on Networking*. https://doi.org/10.1109/TNET.2022.3206781

Patel, J., Buhl-Nielsen, L. G., Ghosn, A., & Kogias, M. (2026). KRAKENGUARD: Towards fine-grained eBPF isolation. *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*.

Peng, T., Lin, C.-H., Wei, Y.-C., Chen, B.-C., & Yeh, J.-Y. (2026). Low-overhead passive resilience monitoring for AIoT gateways: End-to-end anomaly detection via eBPF timing traces and autoencoders. *IEEE Access*. https://doi.org/10.1109/ACCESS.2026.3685156

Remya, S., Pillai, M. J., Niranjan, B., Kumar, P. M. A., Shaju, K. M., Raj, K. D., Ramasubbareddy, S., & Cho, Y. (2025). CODAX: eBPF-based runtime detection of semantic DDoS attacks in Linux containers. *IEEE Access*, *13*. https://doi.org/10.1109/ACCESS.2025.3614389

Repetto, M., & Carrega, A. (2022). Monitoring network flows in containerized environments. In *Lecture notes in computer science* (Vol. 13300). Springer. https://doi.org/10.1007/978-3-031-04036-8_2

Rohliyanto, A., & Utami, E. (2026). Evaluasi efektivitas firewall pre-filtering berbasis eBPF/XDP menggunakan Random Forest untuk deteksi anomali trafik pada Docker Swarm [Evaluation of the effectiveness of eBPF/XDP-based firewall pre-filtering using Random Forest for traffic anomaly detection on Docker Swarm]. *JURIKOM (Jurnal Riset Komputer)*, *13*(2). https://doi.org/10.30865/jurikom.v13i2.9618

Ryu, J., Kim, R., Lee, S., Kim, S., Choi, H., & Kim, S. (2026). Hybrid runtime detection of malicious containers using eBPF. *Computers, Materials & Continua*, *82*(3). https://doi.org/10.32604/cmc.2025.074871

Sadiq, A., Syed, H. J., Ansari, A. A., Ibrahim, A. O., Alohaly, M., & Elsadig, M. (2023). Detection of denial of service attack in cloud based Kubernetes using eBPF. *Applied Sciences*, *13*(8), 4700. https://doi.org/10.3390/app13084700

Shin, H., Jo, M., Yoo, H., Lee, Y., & Tak, B. (2024). A technique for accurate detection of container attacks with eBPF and AdaBoost. *Journal of the Korea Society of Computer and Information*, *29*(6), 39–51. https://doi.org/10.9708/jksci.2024.29.06.039

Soldani, D., Monaco, F., Nahi, P., Risso, F., Soliman, M. F., Di Giovanna, L., Jafarizadeh, S., Ognibene, G., & Bour, H. (2023). eBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond). *IEEE Access*, *11*. https://doi.org/10.1109/ACCESS.2023.3281480

Sun, H., & Su, Z. (2024). Validating the eBPF verifier via state embedding. *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*.

Syairozi, A. A., & Arizal. (2025). Comparative analysis of eBPF-based runtime security monitoring tools in monitoring and threat

detection on Kubernetes. *Proceedings of RITECH 2025*. https://doi.org/10.5220/0014272700004928

Taguchi, H., Hara, T., & Kasahara, S. (2026). Unsupervised real-time in-kernel intrusion detection system using autoencoders and eBPF. *IEICE Transactions on Communications*, *E109.B*(1). https://doi.org/10.23919/transcom.2025EBP3028

Takagaki, T., & Mizutani, K. (2025). Deep analysis of containerized web server performance using eBPF-captured data. *IEEJ Transactions on Electrical and Electronic Engineering*. https://doi.org/10.1002/tee.70070

Wang, H., Yu, A., Xiao, L., & Meng, D. (2026). Vaccine: Injection vulnerabilities mitigation through dynamic process control with eBPF. *Computers & Security*, *162*, 104788. https://doi.org/10.1016/j.cose.2025.104788

Wang, S.-Y., & Chang, J.-C. (2022). Design and implementation of an intrusion detection system by using extended BPF in the Linux kernel. *Journal of Network and Computer Applications*, *198*, 103283. https://doi.org/10.1016/j.jnca.2021.103283

Yang, S., Kang, B. B., & Nam, J. (2024). Optimus: Association-based dynamic system call filtering for container attack surface reduction. *Journal of Cloud Computing*. https://doi.org/10.1186/s13677-024-00639-3

Yang, W., Chen, P., Yu, G., Zhang, H., & Zhang, H. (2024). Network shortcut in data plane of service mesh with eBPF. *Journal of Network and Computer Applications*, *222*, 103805. https://doi.org/10.1016/j.jnca.2023.103805

Yoon, G., Shin, J., Kim, J.-S., Kim, S., Jeong, J., & Choi, Y.-H. (2026). Container-specific service mesh-based system for mitigating lateral movement attacks. *IEEE Transactions on Cloud Computing*, *14*(1), 92–109. https://doi.org/10.1109/TCC.2026.3657486

You, M.-s., Kim, J.-W., Shin, S., & Park, T. (2022). BPFast: An eBPF/XDP-based high-performance packet payload inspection system for cloud environments. *Journal of the Korea Institute of Information Security and Cryptology*, *32*(2), 213–225. https://doi.org/10.13089/JKIISC.2022.32.2.213

Zehra, S., Syed, H. J., Samad, F., & Faseeha, U. (2025). DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time. *IEEE Access*, *13*. https://doi.org/10.1109/ACCESS.2025.3592192

Zhan, M., Li, Y., Yang, H., Yu, G., Li, B., & Wang, W. (2023). Coda: Runtime detection of application-layer CPU-exhaustion DoS attacks in containers. *IEEE Transactions on Services Computing*, *16*(4). https://doi.org/10.1109/TSC.2022.3194266

Zhong, Y., Chen, P., & Zhang, H. (2024). ESX: A self-generated control policy for remote access with SSH based on eBPF. *IEEE Access*. https://doi.org/10.1109/ACCESS.2024.3450496

Zhou, N., Chen, H., Chen, Z., Li, C., & Li, F. (2026). An abnormal file access detection model for containers based on eBPF listening. *Mathematics*, *14*(6), 991. https://doi.org/10.3390/math14060991